PHASE 1 — DEEP DIVE

Advanced JavaScript, TypeScript, React & Next.js

Month 1–2 · Every resource listed with exact video/playlist title + channel name

AI Product Engineer Master Roadmap — Companion Document

Phase Duration	8 weeks (Month 1 and 2 of the full roadmap)
Daily Commitment	2–3 hours weekdays, 4–6 hours weekends (adjust to your pace)
Your Starting Point	2 years frontend exp — JS, React, HTML, CSS. You know the basics.
Goal of This Phase	Close every gap in JS internals, TypeScript, React depth, Next.js, performance & testing
Why This Matters for AI	LLM streaming UIs, token rendering, multi-provider abstraction, real-time updates — all require deep frontend mastery
End State	You can build a production-grade AI streaming chat app with TypeScript, tests, deployed to Vercel
Total Topics	8 core topics — each with 2–4 weeks of focused work, multiple resources, and a project
Capstone Project	AI Streaming Chat Application — Portfolio Project #1

TOPIC 1 — Advanced JavaScript Internals · *Weeks 1–2 · Foundation of everything*

You know JavaScript syntax. This topic makes you understand how JavaScript actually works — the engine, the runtime, closures, prototypes, and asynchronous internals. This is non-negotiable. AI streaming UIs require deep async knowledge. Debugging memory leaks in LLM applications requires knowing the runtime cold.

1.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Playlist	Namaste JavaScript — Season 1 (Full Playlist, 18 videos)	Akshay Saini	YouTube → Search 'Namaste JavaScript Akshay Saini'	THE best JS internals course in existence. Covers execution context, call stack, hoisting, closure, scope chain, event loop — all visually explained. Free.
2	Playlist	Namaste JavaScript — Season 2 (Full Playlist, 10 videos)	Akshay Saini	YouTube → Search 'Namaste JavaScript Season 2'	Covers promises, async/await, generators, iterators. Watch after Season 1. Essential for LLM streaming.
3	Video	JavaScript Visualized: Event Loop	Lydia Hallie	YouTube → Search 'JavaScript Visualized Event Loop Lydia Hallie'	5-minute visual animation of the event loop. Watch this FIRST — sets the mental model before Akshay's deep dive.
4	Video	JavaScript Visualized: Promises & Async/Await	Lydia Hallie	YouTube → Search 'JavaScript Visualized Promises Lydia Hallie'	Best visual explainer of promises. Complements Season 2 of Namaste JS.
5	Video	JavaScript Closures Explained in 10 Minutes	Web Dev Simplified	YouTube → Search 'JavaScript Closures Web Dev Simplified'	Clean, practical closure examples. Watch alongside Akshay's closure video.
6	Video	Prototypal Inheritance in JavaScript	Fireship	YouTube → Search 'Prototypal Inheritance JavaScript Fireship'	7-minute explainer. Best quick reference for prototype chain.
7	Video	JavaScript Generators — Lazy Evaluation	Fun Fun Function	YouTube → Search 'Generators JavaScript Fun Fun Function'	Generators power streaming — this video makes them click.
8	Book	You Don't Know JS — Scope & Closures (free)	Kyle Simpson (getify)	GitHub → search 'You Dont Know JS getify' → read Scope & Closures + Async & Performance	Free on GitHub. Read chapters alongside videos. Most thorough JS reference available.
9	Book	You Don't Know JS — Async & Performance (free)	Kyle Simpson (getify)	GitHub → 'You Dont Know JS' repo → Async & Performance	Read the Promises and Generators chapters. Foundational for AI streaming.

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
10	Website	javascript.info — Part 1: The Language	javascript.info	Visit javascript.info → The JavaScript language section	Best written JS reference. Use as reference when a concept is unclear after videos.

1.2 — What to Study (Topic Breakdown)

Execution Context & Call Stack

- How JavaScript creates execution contexts (global + function)
- The call stack — how functions are pushed and popped
- Variable environments and the scope chain
- Hoisting — why var/let/const behave differently

> ⍰ Watch: 'Namaste JavaScript Ep 1–4' by Akshay Saini — these 4 videos alone fix 80% of JS confusion that developers carry for years.

Closures — Deep Understanding

- What a closure is and why it exists (lexical environment)
- Closure in loops — the classic var vs let problem
- Module pattern using closures
- Memory implications — when closures cause leaks in long-running AI apps

> ⍰ Watch: 'Namaste JavaScript Ep 10–13' (Closures series). Then immediately write 3 closure examples from scratch without references.

Event Loop, Macrotask & Microtask Queue

- The single-threaded model — why JS can be non-blocking
- Microtask queue (Promises, queueMicrotask) vs Macrotask queue (setTimeout, setInterval)
- How async/await desugars to Promise chains
- Why LLM token streaming requires understanding the event loop

> ⍰ Watch: 'Namaste JavaScript Ep 15–18' (Event Loop series). This is the most important topic for AI UI development.

Promises & Async/Await Internals

- Promise states: pending, fulfilled, rejected
- Promise chaining — .then(), .catch(), .finally()
- Promise combinators: Promise.all(), Promise.race(), Promise.allSettled(), Promise.any()
- Async/await — syntactic sugar over promises (know the desugared version)
- Error handling patterns — try/catch vs .catch()

⍰ Build from scratch: Implement Promise.all() and Promise.race() without using the built-in versions. If you can do this, you understand promises.

Generators & Iterators (critical for AI streaming)

- Iterator protocol — Symbol.iterator
- Generator functions — function* and yield
- Async generators — async function* — this is HOW LLM streaming is implemented internally
- ReadableStream — the browser API that wraps async generators for HTTP streaming

⍰ Watch: 'Namaste JavaScript Season 2, Ep 8–10'. Then read the Vercel AI SDK source code — it uses async generators internally.

1.3 — Stop Conditions (JavaScript Mastery Checklist)

You have mastered JavaScript internals when you can check ALL of these without looking anything up:

☐ Explain the difference between execution context, scope, and closure in one whiteboard diagram
☐ Implement a debounce function from scratch (no library)
☐ Implement a Promise from scratch that passes basic tests
☐ Implement Promise.all() from scratch
☐ Explain why microtasks run before macrotasks with a working code example
☐ Write an async generator that yields values from a fake streaming API
☐ Explain what happens in memory when a closure captures a large object
☐ Debug an event loop starvation issue in a React app that renders LLM tokens
☐ Explain the prototype chain of a class instance
☐ Implement the module pattern using closures and IIFEs

TOPIC 2 — TypeScript (Strict Mode & Advanced Types) · *Weeks 2–3 · Non-negotiable for production AI SaaS*

TypeScript is not optional. Every serious AI SaaS company uses TypeScript in strict mode. AI applications have complex data shapes — LLM responses, tool call results, agent states — and TypeScript prevents entire categories of bugs at compile time. This topic takes you from 'I know basic TypeScript' to 'I write TypeScript that scales'.

2.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	TypeScript Course for Beginners — Full Course 2023 (3.2 hrs)	Academind (Maximilian Schwarzmüller)	YouTube → Search 'TypeScript Full Course Academind 2023'	Best intro-to-intermediate TypeScript course. Covers all fundamentals systematically.
2	Playlist	TypeScript Tips (ongoing series, 50+ short videos)	Matt Pocock (Total TypeScript)	YouTube → Search 'Matt Pocock TypeScript Tips' OR visit totaltypescript.com	The #1 advanced TypeScript resource. Each video is 3–8 min and teaches one specific pattern. Watch from the beginning.
3	Video	TypeScript Generics Explained	Web Dev Simplified	YouTube → Search 'TypeScript Generics Explained Web Dev Simplified'	Clearest generics explainer for beginners. Watch before Matt Pocock's generics content.
4	Video	TypeScript - The Basics (15 min)	Fireship	YouTube → Search 'TypeScript the Basics Fireship'	Great 15-minute overview to warm up. Watch first if TS is relatively new to you.
5	Video	Zod — TypeScript-first Schema Validation (14 min)	Web Dev Simplified	YouTube → Search 'Zod Tutorial Web Dev Simplified'	Zod is essential for validating LLM outputs. Learn it alongside TypeScript.
6	Video	Advanced TypeScript Types — Utility Types Explained	Matt Pocock	YouTube → Search 'Advanced TypeScript Utility Types Matt Pocock'	Covers Partial, Required, Pick, Omit, ReturnType, Parameters — used constantly in AI apps.
7	Playlist	TypeScript Challenges (type-challenges walkthrough)	Various — search on YouTube	YouTube → Search 'type-challenges solutions walkthrough'	type-challenges on GitHub is the LeetCode of TypeScript. Do the Easy and Medium challenges.
8	Website	Total TypeScript — Free Tutorials	Matt Pocock	totaltypescript.com → Free section	Interactive exercises in the browser. Best way to practice

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
					advanced TypeScript patterns.
9	Docs	The TypeScript Handbook (official)	Microsoft / TypeScript Team	typescriptlang.org/docs/handbook	Read: Narrowing, Generics, Template Literal Types, Conditional Types, Utility Types sections.

2.2 — What to Study

Type System Fundamentals

- Primitive types, arrays, tuples, enums
- Interfaces vs Type aliases — when to use each
- Union types (A | B) and Intersection types (A & B)
- Literal types — const narrowing
- The 'unknown' type vs 'any' — why unknown is safer for LLM responses

Narrowing & Type Guards

- typeof, instanceof, in operator for narrowing
- Custom type predicates (x is Type)
- Discriminated unions — the pattern used in LLM tool call types
- Exhaustiveness checking with 'never'

> 📺 Watch: Matt Pocock's 'Discriminated Unions' video. This exact pattern is how OpenAI tool calls are typed in the SDK.

Generics — The Most Important Advanced Topic

- Generic functions, generic interfaces, generic classes
- Constraints with 'extends'
- Default generic parameters
- Infer keyword in conditional types
- Practical: Type a generic API response wrapper used for all LLM API calls

Utility Types (used constantly in AI apps)

- Partial<T>, Required<T>, Readonly<T>
- Pick<T,K>, Omit<T,K>, Exclude<T,U>, Extract<T,U>
- ReturnType<T>, Parameters<T>, InstanceType<T>
- Record<K,V> — for typed LLM response maps
- NonNullable<T> — for stripping null from LLM optional fields

Strict Mode & Configuration

- tsconfig.json — strict: true and what it enables
- strictNullChecks — eliminates entire class of runtime bugs
- noImplicitAny — forces you to type everything properly
- exactOptionalPropertyTypes — stricter optional handling

> ⍰ Rule: Never use 'any'. If you are tempted to use 'any', use 'unknown' and narrow it. This single discipline makes you a 10x better TypeScript developer.

2.3 — Stop Conditions (TypeScript Mastery Checklist)

☐ Migrate a real JavaScript project to TypeScript strict mode with zero 'any'
☐ Write a generic API response wrapper: ApiResponse<T> that handles success and error states
☐ Implement a discriminated union for LLM message types (user | assistant | system | tool)
☐ Use 'infer' in a conditional type to extract the resolved type of a Promise
☐ Write a custom type predicate for runtime type checking of LLM tool call results
☐ Configure tsconfig.json for a Next.js project with all strict options enabled
☐ Use Zod to validate an OpenAI API response and infer the TypeScript type from the schema
☐ Implement all 5 most common utility types (Partial, Pick, Omit, ReturnType, Record) from scratch

TOPIC 3 — React Internals & Deep Concepts · *Weeks 3–4 · Concurrent rendering for AI UIs*

You know React. This topic makes you understand React at the engine level — Fiber, reconciliation, concurrent features, hooks internals. This knowledge is directly applied when building AI UIs: streaming responses cause many small re-renders, and understanding React's batching and scheduling prevents UI freezes.

3.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Playlist	React 18 Deep Dives — Full Playlist (10 videos)	Jack Herrington	YouTube → Search 'React 18 Deep Dives Jack Herrington' — look for playlist	Most technical React series on YouTube. Covers Fiber, Suspense, transitions, concurrent rendering. Essential.
2	Video	React Fiber Architecture Explained	Theo (t3.gg)	YouTube → Search 'React Fiber Architecture Theo t3gg'	Best plain-English explanation of React Fiber. Watch before Jack's deep dives.
3	Video	When Does React Re-render? (Full Explanation)	Jack Herrington	YouTube → Search 'When Does React Re-render Jack Herrington'	This video alone will fix 90% of React performance bugs you currently make.
4	Video	useCallback and useMemo — When to Actually Use Them	Web Dev Simplified	YouTube → Search 'useCallback useMemo Web Dev Simplified'	Most developers misuse these. Clear explanation of when they help vs hurt.
5	Video	React's New use() Hook and Suspense for Data Fetching	Theo (t3.gg)	YouTube → Search 'React use hook Suspense Theo t3gg'	Critical for AI streaming UIs — Suspense boundaries for async LLM responses.
6	Video	Build Your Own useState and useEffect from Scratch	Web Dev Simplified	YouTube → Search 'Build useState useEffect from scratch Web Dev Simplified'	Implementing hooks from scratch is the fastest way to understand them deeply.
7	Video	React Context vs Zustand vs Redux — Full Comparison	Theo (t3.gg)	YouTube → Search 'React Context vs Zustand Theo t3gg'	Clears up state management confusion permanently. Watch before choosing state tools.
8	Video	React Reconciliation — How the Diffing Algorithm Works	Traversy Media	YouTube → Search 'React Reconciliation Algorithm Traversy Media'	Visual walkthrough of how React decides what to re-render.
9	Docs	React.dev — Reference section (official new docs)	React Core Team	react.dev → Reference section → Hooks reference	New official docs are excellent. Read the full API reference for useState, useEffect,

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
					useRef, useCallback, useMemo, useReducer.
10	Video	React Compiler Explained — What It Does to Your Code	Theo (t3.gg)	YouTube → Search 'React Compiler Explained Theo'	React Compiler (React Forget) is the future — understand what it automates.

3.2 — What to Study

React Fiber — the Reconciler

- What is Fiber? — The internal unit of work in React
- Why React rewrote the reconciler from a stack to a fiber-based architecture
- Work loop, work-in-progress tree, current tree
- How React pauses, resumes, and abandons work (concurrent features)

> ⍰ You don't need to read the Fiber source code. The goal is a mental model: React builds a 'to-do list' of DOM updates and can pause/resume it. This is why Suspense and transitions work.

Reconciliation & Diffing

- The diffing algorithm — O(n) heuristics React uses
- Why 'key' prop is critical for lists — and what happens when you get it wrong
- Reconciliation rules: same type = update, different type = remount
- How React batches multiple setState calls into one re-render (React 18 automatic batching)

Hooks Internals

- How hooks are stored in a linked list on the fiber node
- Why hooks cannot be called conditionally (the rules of hooks explained)
- useState — the actual implementation (queue of updates)
- useEffect — cleanup, dependencies array mechanics, why stale closures happen
- useRef — how it bypasses the re-render cycle
- Custom hooks — composition pattern for AI features (useStreamingMessage, useChatHistory)

> ⍰ Build these from scratch: useState, useEffect, useCallback, useMemo. Implement them using a global array. This exercise produces permanent understanding.

Concurrent Features (React 18) — Critical for AI UIs

- Concurrent rendering — React can render in the background
- startTransition — mark non-urgent state updates (e.g., LLM token appending)
- useDeferredValue — defer expensive renders while streaming AI tokens
- Suspense — declarative loading boundaries for async AI data
- Automatic batching — all state updates batched by default in React 18

> ⍰ AI chat apps render thousands of small token updates. Using startTransition for token rendering prevents UI jank — this is production knowledge most developers lack.

3.3 — Stop Conditions (React Internals Checklist)

☐ Implement useState from scratch using a global array — works correctly for multiple hooks
☐ Implement useEffect with cleanup and dependency comparison from scratch
☐ Explain the difference between a re-render and a remount — give an example of each
☐ Fix a stale closure bug in a useEffect that uses a callback from props
☐ Use startTransition to prevent LLM token rendering from blocking user input
☐ Build a custom useStreamingMessage hook that subscribes to a ReadableStream
☐ Explain why key={index} breaks animations in lists and what to use instead
☐ Profile a React component tree with React DevTools Profiler — find and fix a slow render

TOPIC 4 — State Management (Zustand + TanStack Query) · *Week 4 · Modern patterns for AI app state*

AI applications have complex state: active conversations, streaming messages, user preferences, cached LLM responses, server state from multiple endpoints. This topic covers the modern tools that handle this elegantly — Zustand for client state, TanStack Query for server/AI state.

4.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Zustand Tutorial for Beginners (20 min)	Lama Dev	YouTube → Search 'Zustand Tutorial Beginners Lama Dev'	Best intro to Zustand. Covers store creation, slices, devtools, persistence.
2	Video	Zustand vs Redux — Why I Switched (12 min)	Theo (t3.gg)	YouTube → Search 'Zustand vs Redux Theo t3gg'	Explains WHY Zustand is better for modern apps. Sets the right mindset.
3	Video	TanStack Query v5 — Full Tutorial (1.5 hrs)	Lama Dev	YouTube → Search 'TanStack Query v5 Full Tutorial Lama Dev'	Comprehensive TanStack Query v5 course. Covers queries, mutations, optimistic updates, infinite scroll.
4	Video	TanStack Query — Why It's the Best State Manager	Theo (t3.gg)	YouTube → Search 'TanStack Query Best State Manager Theo'	Mental model video — understand what problem TanStack Query solves before diving into API.
5	Video	Optimistic Updates with TanStack Query (10 min)	Jack Herrington	YouTube → Search 'Optimistic Updates TanStack Query Jack Herrington'	Optimistic UI for AI apps — show the message immediately before the LLM responds.
6	Video	React Query Patterns — Dependent Queries, Parallel Queries	Jack Herrington	YouTube → Search 'React Query Patterns Jack Herrington'	Advanced patterns for complex AI data fetching scenarios.
7	Docs	Zustand Official Documentation	pmndrs team	zustand.docs.pmnd.rs — read Getting Started + Guides	Short, excellent docs. Read all of them — under 30 minutes.
8	Docs	TanStack Query v5 Official Docs	TanStack team	tanstack.com/query/v5/docs — read all of Overview + Guides	Best query library docs. Read the Guides section thoroughly.

4.2 — What to Study

Zustand — Client State

- Creating stores with slices pattern
- Zustand devtools integration (Redux DevTools extension)
- Persist middleware — save conversation history to localStorage
- Immer middleware — immutable state updates for complex nested state
- Computed values and subscriptions

> ⍰ For AI apps: use Zustand to store the active conversation, UI state (streaming: boolean, selectedModel: string), and user preferences. Keep it minimal — don't put server data in Zustand.

TanStack Query — Server & AI State

- useQuery — fetch and cache LLM responses, conversation history
- useMutation — post messages, trigger AI tasks
- Query invalidation — refresh conversation after AI response completes
- Optimistic updates — show user message immediately
- Stale-while-revalidate — the caching strategy that TanStack Query is built on
- Background refetching — keep conversation list fresh

> ⍰ Golden rule: Server state (API data, AI responses) goes in TanStack Query. UI state (is sidebar open, selected tab) goes in Zustand. Local component state (form inputs) goes in useState. This separation prevents 90% of state bugs.

4.3 — Stop Conditions

☐ Build a Zustand store for an AI chat app: conversations, messages, streaming state, selected model
☐ Implement the slice pattern — separate stores for auth, conversations, and UI state
☐ Use TanStack Query to fetch conversation history with proper stale time and caching
☐ Implement an optimistic update: show user message immediately, rollback on API error
☐ Persist conversation history to localStorage using Zustand persist middleware
☐ Use useQuery + useMutation together for a send-message flow with loading/error states

TOPIC 5 — Next.js App Router, RSC & Server Actions · *Weeks 5–6 · The standard for AI SaaS frontends*

Most AI SaaS products are built on Next.js. The App Router (Next.js 13+) introduced React Server Components, Server Actions, and edge runtime — all of which change how AI apps are structured. This is the framework you will use for every AI project in this roadmap.

5.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Next.js 14 Full Course 2024 — App Router (4 hrs)	Dave Gray	YouTube → Search 'Next.js 14 Full Course Dave Gray 2024'	Most complete App Router course. Covers routing, layouts, loading.tsx, error.tsx, data fetching, Server Actions.
2	Video	Next.js App Router — Every Feature Explained	Theo (t3.gg)	YouTube → Search 'Next.js App Router Every Feature Theo t3gg'	Mental model video from Theo — understand WHY each App Router feature exists before learning HOW to use it.
3	Video	React Server Components — The Basics (20 min)	Jack Herrington	YouTube → Search 'React Server Components Basics Jack Herrington'	Best plain-English RSC explainer. Watch before the Next.js course.
4	Video	Server Actions in Next.js 14 — Full Guide	Josh tried coding	YouTube → Search 'Server Actions Next.js 14 Josh tried coding'	Josh's tutorials are the gold standard for practical Next.js. This video covers Server Actions completely.
5	Video	Next.js with Vercel AI SDK — Streaming Tutorial	Vercel (official channel)	YouTube → Vercel official YouTube → Search 'AI SDK streaming tutorial'	Official Vercel tutorial for AI SDK. Directly shows how to stream LLM responses in Next.js App Router.
6	Video	Understanding RSC, SSG, SSR, ISR in Next.js	Lee Robinson (Vercel VP)	YouTube → Search 'RSC SSG SSR ISR Next.js Lee Robinson'	Lee (Vercel VP Dev Experience) explains all rendering modes. Critical for AI app architecture decisions.
7	Video	Next.js Caching — Complete Guide (Cache is Broken)	Theo (t3.gg)	YouTube → Search 'Next.js Caching Theo t3gg'	Next.js caching is confusing. This video explains all 4 caching mechanisms clearly.
8	Video	Next.js Authentication with Auth.js (Next-Auth v5)	Josh tried coding	YouTube → Search 'Next.js Authentication Auth.js Josh tried coding'	Auth is required for AI SaaS. Best tutorial for Auth.js (NextAuth) v5 in App Router.

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
9	Docs	Next.js Official Documentation	Vercel Team	nextjs.org/docs — read App Router section completely	Next.js docs are excellent. Read: Routing, Data Fetching, Server Components, Server Actions, Optimizing.
10	Blog	Josh Comeau — Understanding React Server Components	Josh Comeau	joshwcomeau.com → search 'Understanding React Server Components'	Best written explainer for RSC. Read this before or after the video resources.

5.2 — What to Study

App Router Architecture

- File-based routing: page.tsx, layout.tsx, loading.tsx, error.tsx, not-found.tsx
- Route groups — (auth), (dashboard) — organise without affecting URL
- Parallel routes and intercepting routes
- Metadata API — SEO for AI SaaS landing pages

React Server Components (RSC)

- Server Components run on the server — can access DB, secrets, file system directly
- Client Components need 'use client' directive — can use hooks, event handlers
- Composition pattern: Server Component wrapping Client Components
- Data fetching in Server Components — async/await directly (no useEffect needed)

> 💡 Mental model: Server Components are like PHP pages — they run once, render HTML, and send it. Client Components are like classic React — they run in the browser and are interactive. Compose them strategically.

Server Actions — The Key to AI SaaS

- Server Actions are async functions that run on the server — called from the client
- Use for: sending messages, creating conversations, triggering AI tasks
- Streaming from Server Actions — return a ReadableStream for token-by-token responses
- Form actions vs button onClick with Server Actions
- Error handling and progressive enhancement

> 💡 Server Actions + Vercel AI SDK = the most elegant way to stream LLM responses in Next.js. The AI SDK's streamText() function works perfectly with Server Actions. This is what you use in your capstone project.

Rendering Strategies for AI Apps

- Static Generation (SSG) — marketing pages, blog posts
- Server-Side Rendering (SSR) — dashboard pages that need fresh data
- Incremental Static Regeneration (ISR) — pages that update periodically

- Client-Side Rendering (CSR) — when to use for AI chat interfaces
- Edge Runtime — run Next.js routes at CDN edge nodes (lower latency for AI APIs)

5.3 — Stop Conditions

☐ Build a Next.js App Router app with auth, DB connection, and streaming Server Action — deployed to Vercel
☐ Explain the difference between Server Components and Client Components with a diagram
☐ Use Server Actions to stream an OpenAI response token-by-token to the client
☐ Implement route groups for (auth) and (dashboard) with different layouts
☐ Configure dynamic metadata for a multi-page AI SaaS app
☐ Use Suspense boundaries to show loading states while AI data fetches
☐ Explain all 4 Next.js caching layers and when each one is active

TOPIC 6 — Frontend Performance Optimization · *Week 6 · AI UIs have unique performance challenges*

AI streaming interfaces render thousands of small updates. Without performance optimisation, token streaming causes UI jank, high CPU usage, and dropped frames. This topic covers Core Web Vitals, React profiling, and the specific optimisations needed for AI applications.

6.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Web Performance Fundamentals — LCP, CLS, INP Explained	Google Chrome Developers (official)	YouTube → Search 'Core Web Vitals Explained Google Chrome Developers'	Official Google video explaining all Core Web Vitals with real examples.
2	Video	React Performance — How to Avoid Unnecessary Re-renders	Jack Herrington	YouTube → Search 'React Performance Unnecessary Re-renders Jack Herrington'	Most practical React performance video. Directly applicable to AI token streaming.
3	Video	Chrome DevTools Performance Panel — Full Tutorial	Google Chrome Developers	YouTube → Search 'Chrome DevTools Performance Panel Tutorial Google'	Learn to use the Performance profiler — essential for diagnosing AI UI slowness.
4	Video	React DevTools Profiler — Find Performance Issues	Ben Awad	YouTube → Search 'React DevTools Profiler Ben Awad'	Best tutorial for React-specific profiling. Shows exactly how to find slow components.
5	Video	Next.js Bundle Analyzer — Reduce Bundle Size	Fireship	YouTube → Search 'Next.js Bundle Analyzer Fireship'	Identify and eliminate large dependencies that slow down AI app load time.
6	Video	Virtualization with TanStack Virtual — Render 100K Items	Jack Herrington	YouTube → Search 'TanStack Virtual Jack Herrington'	Long conversation histories need virtualisation. This video shows how.
7	Video	Image Optimization in Next.js — The Complete Guide	Josh tried coding	YouTube → Search 'Next.js Image Optimization Josh tried coding'	next/image is critical for AI apps with user-uploaded images.
8	Guide	web.dev Performance Guide (Google)	Google Web Developers	web.dev/performance — read the Core Web Vitals section	Best written performance reference. Bookmark and use as reference during debugging.

6.2 — AI-Specific Performance Patterns

Streaming Token Rendering Optimization

- Problem: Each LLM token causes a React state update → re-render → potential jank
- Solution 1: Batch token updates with requestAnimationFrame before setState
- Solution 2: Use startTransition to mark token updates as non-urgent
- Solution 3: useDeferredValue for the message list during active streaming
- Solution 4: Memoize non-streaming messages with React.memo

> ⍰ Rule: During active LLM streaming, only the current streaming message should be in 'hot' state. All previous messages should be memoized and never re-render during streaming.

Conversation History Performance

- Problem: Long conversation histories (100+ messages) cause slow renders
- Solution: TanStack Virtual — only render visible messages in the viewport
- Code splitting: lazy-load chat history components
- Pagination vs infinite scroll for conversation loading

Core Web Vitals for AI SaaS

- LCP (Largest Contentful Paint) — must be <2.5s — use Next.js ISR for landing pages
- CLS (Cumulative Layout Shift) — AI responses that push content down = bad CLS
- INP (Interaction to Next Paint) — user input during streaming must feel instant

6.3 — Stop Conditions

☐ Achieve LCP<2.5s, CLS<0.1, INP<200ms on your AI Chat app (PageSpeed Insights score >90)
☐ Profile a slow React component tree and eliminate unnecessary re-renders during streaming
☐ Implement virtualisation for a conversation with 500+ messages using TanStack Virtual
☐ Use startTransition to prevent LLM token appending from blocking keyboard input
☐ Analyse bundle size with @next/bundle-analyzer — ensure no single chunk >250KB
☐ Implement lazy loading for non-critical AI features (image generation UI, etc.)

TOPIC 7 — Testing (Vitest + Testing Library + Playwright) · *Week 7 · AI outputs are non-deterministic — tests are essential*

AI applications are uniquely hard to test — LLM outputs are non-deterministic. This makes a strong test foundation even more critical. You need to mock streaming responses, test component behaviour with fake AI outputs, and write E2E tests that verify the full user flow.

7.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Vitest — The Complete Guide (1 hr)	Web Dev Simplified	YouTube → Search 'Vitest Complete Guide Web Dev Simplified'	Best Vitest introduction. Covers test runner, mocking, coverage, TypeScript support.
2	Playlist	React Testing Library Tutorial (playlist, 8 videos)	The Net Ninja	YouTube → Search 'React Testing Library Tutorial The Net Ninja' → open playlist	Best step-by-step React Testing Library course. Covers queries, events, async testing.
3	Video	Testing React Hooks — The Right Way	Jack Herrington	YouTube → Search 'Testing React Hooks Jack Herrington'	Covers testing custom hooks — needed for testing useStreamingMessage.
4	Video	Playwright Tutorial — End-to-End Testing (1.5 hrs)	Traversy Media	YouTube → Search 'Playwright Tutorial End to End Testing Traversy Media'	Best Playwright intro. Covers writing, running, and debugging E2E tests.
5	Video	How to Test AI Features — Mocking LLM Responses	Kevin Powell	YouTube → Search 'Testing AI Features Mocking LLM Kevin Powell'	Specific to AI app testing — how to mock streaming responses.
6	Video	MSW (Mock Service Worker) — API Mocking for Tests	Jack Herrington	YouTube → Search 'MSW Mock Service Worker Jack Herrington'	MSW is the best tool for mocking OpenAI API calls in tests.
7	Video	Test Coverage in Vitest — What to Actually Test	Theo (t3.gg)	YouTube → Search 'Test Coverage Vitest Theo t3gg'	Pragmatic view on what percentage of coverage to target and what to test.
8	Docs	Testing Library — Core Principles	Testing Library team	testing-library.com/docs/guiding-principles	Read the 'Guiding Principles' page. Understanding WHY Testing Library works the way it does makes every test easier to write.

7.2 — What to Study

Unit Testing with Vitest

- Test file structure: co-locate tests with components (__tests__ folder or .test.tsx)
- Describe blocks, test/it functions, expect assertions
- Mocking: vi.mock(), vi.fn(), vi.spyOn()
- Async tests: await, waitFor, findBy queries
- Snapshot testing — when to use and when to avoid

Testing React Components with Testing Library

- Query priority: getByRole > getByLabelText > getByText > getByTestId
- User interactions: userEvent (preferred over fireEvent)
- Testing async state changes — waitFor, findBy queries
- Testing custom hooks with renderHook

Mocking AI/LLM Responses in Tests

- Mock OpenAI API calls with vi.mock() — return fake streaming responses
- Mock ReadableStream for testing token streaming components
- Use MSW (Mock Service Worker) to intercept HTTP requests at network level
- Test error states: API timeout, rate limit response, malformed JSON

> ⍰ Key insight: You never test that OpenAI returns the 'right' answer — you test that your UI handles responses correctly. Mock the API to return a known response, then assert the UI behaves correctly.

E2E Testing with Playwright

- Page Object Model pattern — abstract page interactions
- Testing the full chat flow: type message → send → receive streaming response
- Screenshot testing for AI UI regression
- CI integration — run Playwright on every pull request

7.3 — Stop Conditions

☐ Write 80%+ test coverage for the AI Chat UI (measured by Vitest coverage report)
☐ Mock a streaming ReadableStream in tests and assert token-by-token rendering
☐ Set up MSW to intercept OpenAI API calls — all tests pass without real API calls
☐ Write Playwright E2E test for full chat flow — type, submit, verify streaming response
☐ Test all error states: network error, API timeout, malformed response
☐ Integrate Playwright into GitHub Actions CI — tests run automatically on every push

TOPIC 8 — Vercel AI SDK (The AI Integration Layer) · *Week 8 · The standard for AI in Next.js*

The Vercel AI SDK is the best abstraction for building AI-powered Next.js applications. It handles streaming, multi-provider support, structured outputs, and React hooks for AI state — all in one library. Every AI project in this roadmap uses it.

8.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Vercel AI SDK 3.0 — Official Introduction	Vercel (official channel)	YouTube → Vercel official YouTube channel → Search 'AI SDK 3.0'	Official intro from the Vercel team. Watch this first — 15 minutes.
2	Video	Build an AI Chatbot with Vercel AI SDK and Next.js	Vercel (official channel)	YouTube → Vercel official YouTube → Search 'AI Chatbot Vercel AI SDK Next.js'	Step-by-step official tutorial. Follow along to build your first streaming AI app.
3	Video	Vercel AI SDK — streamText, streamObject, generateText Explained	Sam Selikoff	YouTube → Search 'Vercel AI SDK streamText streamObject Sam Selikoff'	Explains the three main SDK functions and when to use each.
4	Video	Multi-Model AI App with Vercel AI SDK (GPT, Claude, Gemini)	Jack Herrington	YouTube → Search 'Multi Model AI App Vercel AI SDK Jack Herrington'	Shows how to build provider-agnostic AI features — critical for production apps.
5	Video	AI SDK Tool Calling — Function Calling in Next.js	Vercel (official channel)	YouTube → Vercel YouTube → Search 'AI SDK Tool Calling'	Tool calling is the foundation of AI agents. Official tutorial.
6	Video	useChat and useCompletion Hooks — Full Guide	Delba de Oliveira (Vercel)	YouTube → Search 'useChat useCompletion Vercel AI SDK Delba'	useChat is the main hook for chat interfaces. Deep dive with real examples.
7	Docs	Vercel AI SDK Official Documentation	Vercel Team	sdk.vercel.ai — read Getting Started + API Reference	Best-in-class docs. Read the entire Getting Started section and the API reference for useChat, streamText, generateObject.

8.2 — What to Build

Core SDK Functions to Master

- streamText() — stream text responses from any LLM provider
- generateText() — non-streaming text generation for background tasks
- streamObject() — stream structured JSON objects (for AI forms, extraction)
- generateObject() — generate structured JSON synchronously

React Hooks

- useChat() — complete chat interface state management (messages, input, streaming)
- useCompletion() — single-turn text completion
- useObject() — stream structured objects into your component

Provider Abstraction Pattern

- Configure multiple providers: OpenAI, Anthropic, Google, Mistral, local Ollama
- Switch providers via environment variable — same application code works with any model
- Fallback logic: if GPT-4o fails, try Claude 3.5 Sonnet — transparent to the user

> ⚠ The provider abstraction is the single most important architectural decision in AI SaaS. Build it from the start. The ability to swap models without code changes saves you when providers go down or raise prices.

8.3 — Stop Conditions

☐ Build AI streaming chat using useChat() hook with GPT-4o, Claude 3.5, and Gemini all working
☐ Implement provider fallback: if OpenAI returns 429, automatically retry with Anthropic
☐ Use streamObject() to stream structured JSON from an LLM into a real-time updating form
☐ Add tool calling: define a weather tool and a web search tool — LLM calls them correctly
☐ Token counting: display live token usage from AI SDK metadata during streaming

📅 Phase 1 — Week-by-Week Schedule (8 Weeks)

This is your exact week-by-week plan. Each week has one primary topic, specific videos to watch, a hands-on task, and a clear done condition. Follow this strictly — deviate only with good reason.

Week	Topic Focus	Exact Videos to Watch	Hands-On Task	Done When…
Week 1	JS Internals (Part 1)	Namaste JS S1 Ep 1–9 (Akshay Saini) Event Loop — Lydia Hallie	Build: Promise implementation from scratch Build: Event emitter from scratch	Can explain call stack, hoisting, closure, scope chain without notes
Week 2	JS Internals (Part 2) + TS Basics	Namaste JS S1 Ep 10–18 + S2 Ep 1–5 TS Full Course — Academind (first 2 hrs)	Build: Promise.all + Promise.race from scratch Migrate a small JS module to TypeScript	Event loop mastered; basic TypeScript compiling with no errors
Week 3	TypeScript Advanced + TS in React	Matt Pocock TypeScript Tips (first 20 videos) Zod Tutorial — Web Dev Simplified	Rewrite your JS promise library in strict TypeScript Build typed Zod schema for an AI message type	Write discriminated unions, generics, and utility types without docs
Week 4	React Internals + Hooks	React 18 Deep Dives — Jack Herrington (Ep 1–5) React Fiber — Theo (t3.gg)	Implement useState + useEffect from scratch Build useStreamingMessage custom hook	No stale closure bugs; can explain reconciliation verbally
Week 5	State Management + Next.js Basics	Zustand Tutorial — Lama Dev TanStack Query v5 — Lama Dev (first 1 hr) Next.js 14 Full Course — Dave Gray (first 2 hrs)	Build Zustand store for AI chat (conversations, messages, UI state) Connect TanStack Query to a fake API	State is organised correctly; no prop drilling; no server state in Zustand
Week 6	Next.js App Router + Server Actions + Performance	Next.js App Router Every Feature — Theo Server Actions — Josh tried coding Vercel AI SDK Official Intro + Chatbot Tutorial	Build Next.js app with Server Action that streams from OpenAI Measure and optimise Core Web Vitals	Streaming chatbot deployed to Vercel; LCP<2.5s on Lighthouse
Week 7	Testing	Vitest Complete Guide — Web Dev Simplified React Testing Library — The Net Ninja (Ep 1–5) Playwright Tutorial — Traversy Media	Write unit tests for all custom hooks (80%+ coverage) Write Playwright E2E test for full chat flow	Tests pass in CI without real API calls; Playwright test green
Week 8	Capstone Project Build + Polish	Multi-Model AI App — Jack Herrington Zod in Next.js — any reference Vercel AI SDK Tool Calling (official)	Complete the full AI Streaming Chat app Add multi-model support, tool calling, test suite Deploy to Vercel with live URL	Capstone project fully deployed, tested, and documented in GitHub README

⍰ Phase 1 Capstone Project — AI Streaming Chat Application

This is Portfolio Project #1. Build it properly — not as a demo, but as a production-quality application that demonstrates everything you learned in Phase 1. This will be the first project interviewers see when they visit your GitHub.

Full Project Specification

Project Name	ai-chat-streaming (GitHub repo name)
Live Demo Target	Deployed to Vercel — live URL in README. Non-negotiable.
Primary Tech Stack	Next.js 14 App Router + TypeScript (strict) + Tailwind CSS + Shadcn/UI
AI Integration	Vercel AI SDK — streamText() with useChat() hook
Providers Supported	OpenAI (GPT-4o + GPT-4o-mini), Anthropic (Claude 3.5 Sonnet), Google (Gemini 1.5 Pro)
State Management	Zustand (UI state + conversation list) + TanStack Query (server state)
Testing	Vitest + Testing Library (80%+ unit coverage) + Playwright (E2E critical paths)
Performance Target	LCP < 2.5s, CLS < 0.1, INP < 200ms — verified on PageSpeed Insights
TypeScript Rule	Strict mode — zero 'any' types — enforced by ESLint

Required Features (Build All of These)

Core Chat Features

- Token-by-token streaming with smooth rendering (no jank during streaming)
- Markdown rendering with code syntax highlighting (react-markdown + shiki)
- Copy button for code blocks, regenerate response, edit sent message
- Stop streaming button — cancels the ReadableStream mid-response
- Conversation history — persisted to database (Postgres via Prisma)

Multi-Model Support

- Model selector dropdown: GPT-4o, GPT-4o-mini, Claude 3.5 Sonnet, Gemini 1.5 Pro
- Provider abstraction — switching models requires zero code changes in components
- Fallback: if primary model returns 429/503, automatically retry with backup model
- Token counter: show live token usage and estimated cost per message

System Prompt & Personas

- System prompt editor — user can set a custom system prompt
- Pre-built personas: 'Code Expert', 'Writing Assistant', 'Data Analyst'
- Persona settings persisted per conversation in Postgres

UI/UX Polish

- Dark and light mode (Tailwind + next-themes)
- Responsive design: works on mobile, tablet, desktop
- Sidebar: conversation list, new chat button, search conversations
- Keyboard shortcuts: Cmd+Enter to send, Cmd+K for new chat
- Loading states: skeleton screens, typing indicators, streaming cursor

Performance Requirements

- startTransition for token rendering — keyboard input stays responsive during streaming
- React.memo for all non-streaming message components
- TanStack Virtual for conversations with >100 messages
- Next.js Image optimisation for any user-uploaded images

Testing Requirements

- Unit tests: all custom hooks (useStreamingMessage, useConversation, useModelSelector)
- Component tests: ChatMessage, ChatInput, ModelSelector — all states covered
- MSW: mock all OpenAI/Anthropic API calls — tests never call real APIs
- E2E: full conversation flow, multi-model switching, error handling

GitHub README Must Include

- Live demo link (Vercel URL) — at the very top
- Screenshot or GIF of streaming in action
- Architecture diagram: frontend → Server Action → Vercel AI SDK → LLM providers
- 'How streaming works' technical section — your explainer, in your own words
- Tech stack table, local setup instructions (should work in <5 minutes)
- 'What I learned' section — authentic, specific, shows growth

Recommended File Structure

```
src/
├── app/
│   ├── (auth)/login/page.tsx
│   ├── (dashboard)/chat/[id]/page.tsx
│   ├── api/chat/route.ts        ← Vercel AI SDK streaming endpoint
│   └── layout.tsx
├── components/
│   ├── chat/
│   │   ├── ChatInterface.tsx    ← main chat component (client)
│   │   ├── MessageList.tsx      ← virtualised message list
│   │   ├── ChatMessage.tsx      ← individual message with markdown
│   │   └── ChatInput.tsx        ← input + send button
│   └── ui/                      ← Shadcn/UI components
```

```
├── hooks/
│   ├── useStreamingMessage.ts   ← custom streaming hook
│   ├── useConversation.ts       ← conversation state
│   └── useModelSelector.ts      ← model switching logic
├── lib/
│   ├── ai/
│   │   ├── providers.ts         ← provider abstraction (key file)
│   │   └── models.ts            ← model configs + pricing
│   └── db/prisma.ts
├── store/
│   ├── chatStore.ts             ← Zustand UI state
│   └── modelStore.ts
└── __tests__/                   ← all test files
```

Phase 1 Completion Check: You have completed Phase 1 when your AI Streaming Chat App is deployed to Vercel with a live URL, passes all tests in CI, achieves >90 on PageSpeed Insights, and your GitHub README clearly explains the architecture. At that point, you are ready to start Phase 2 — Backend Engineering.

Estimated Time Investment

Week 1–2 (JS Internals)	~20 hrs — 14 hrs videos + 6 hrs building
Week 2–3 (TypeScript)	~18 hrs — 10 hrs videos + 8 hrs hands-on
Week 3–4 (React Internals)	~20 hrs — 12 hrs videos + 8 hrs exercises
Week 4 (State Management)	~12 hrs — 6 hrs videos + 6 hrs project work
Week 5–6 (Next.js + AI SDK)	~22 hrs — 10 hrs videos + 12 hrs building
Week 6 (Performance)	~10 hrs — 4 hrs videos + 6 hrs optimising
Week 7 (Testing)	~16 hrs — 8 hrs videos + 8 hrs writing tests
Week 8 (Capstone polish)	~20 hrs — building, debugging, deploying, documenting
Total Phase 1	~138 hours over 8 weeks ≈ 2.5 hrs/weekday + 5 hrs/weekend day

Critical Warning: Do NOT move to Phase 2 until the capstone project is LIVE on Vercel. A half-built project that lives only on your laptop is worthless for your portfolio. Shipping a slightly imperfect project beats perfecting a project that nobody can see.